Kisi-Kisi UTS Sistem Operasi

Semester 1 · Materi: Pengantar OS, Struktur OS, Proses, IPC, Thread, dan CPU Scheduling

Mata kuliah	: Sistem Operasi
Jenis ujian	: UTS (Ujian Tengah Semester)
Sumber materi	: Silberschatz, *Operating System Concepts* 10th Ed. (Bab 1, 4, 5 dan Bab 2 & Konsep Proses dari materi kuliah)

Cara pakai: baca **Bagian A** untuk tahu cakupan soal, **Bagian B** untuk memahami materi, lalu kerjakan **Bagian C** sebagai latihan. Istilah Inggris penting ada di **Bagian D**.

Daftar Isi

A. Kisi-kisi soal (cakupan dan indikator)
B. Penjelasan materi (dibahas per topik)
C. Latihan soal dan pembahasan (termasuk cara hitung CPU scheduling)
D. Glosarium istilah penting

A. Kisi-Kisi Soal

Tabel ini menunjukkan **apa yang harus bisa kamu jelaskan atau kerjakan** di setiap topik. Kolom "Bentuk soal" adalah perkiraan berdasarkan materi, bukan kepastian.

No	Materi	Indikator (kamu harus bisa…)	Bentuk soal (perkiraan)
1	Pengertian & tujuan OS	Menjelaskan definisi OS, tiga tujuannya, dan peran OS sebagai *resource allocator* dan *control program*	Essay singkat / PG
2	Komponen sistem komputer	Menyebutkan 4 komponen (hardware, OS, aplikasi, user) dan menjelaskan fungsinya	Essay / PG
3	Interrupt & trap	Menjelaskan fungsi interrupt, ISR, interrupt vector, dan perbedaan interrupt dengan trap	Essay
4	Dual mode & timer	Membedakan user mode dan kernel mode, serta menjelaskan cara kerja mode bit dan timer	Essay / PG
5	Hierarki & caching memori	Mengurutkan memori dari tercepat ke terlambat; menjelaskan konsep caching	PG / Essay
6	Batch vs time-sharing	Membedakan multiprogramming dan multitasking (time-sharing)	PG / Essay
7	Layanan & antarmuka OS	Menyebutkan layanan OS untuk user dan untuk efisiensi sistem; membedakan CLI dan GUI	PG / Essay
8	System call	Menjelaskan fungsi system call, API, dan 5 kategorinya; menjelaskan perubahan mode saat system call	Essay

9	Struktur OS	Membandingkan monolitik, layered, microkernel, modular (LKM), dan hybrid (kelebihan, kelemahan, contoh)	Essay / tabel
10	Booting	Mengurutkan tahap booting (BIOS/UEFI → bootloader → kernel → init)	PG / urutan
11	Konsep proses & PCB	Membedakan program dan proses; menyebutkan 5 status proses dan isi PCB	PG / Essay
12	Penjadwal & context switch	Membedakan short-term, long-term, medium-term scheduler; menjelaskan context switch dan overhead-nya	PG / Essay
13	Operasi proses	Menjelaskan fork(), exec(), exit(), wait(), abort(); menjelaskan zombie dan orphan	Essay
14	IPC (komunikasi antar-proses)	Membandingkan shared memory dan message passing; menjelaskan producer-consumer, pipe, socket, RPC	Essay / tabel
15	Thread & multithreading	Menjelaskan komponen thread, beda proses vs thread, dan 4 keuntungan multithreading	PG / Essay
16	Konkurensi & paralelisme	Membedakan concurrency dan parallelism; data vs task parallelism; **menghitung speedup Amdahl**	Essay / **hitungan**
17	Model multithreading	Membedakan user vs kernel thread; menjelaskan many-to-one, one-to-one, many-to-many	Essay / tabel
18	Thread library & implicit threading	Menyebutkan Pthreads, Windows, Java threads; menjelaskan thread pool, OpenMP, GCD	PG / Essay
19	Isu multithreading	Menjelaskan dilema fork(), asynchronous vs deferred cancellation, signal handling, TLS	Essay
20	CPU scheduling – konsep	Membedakan preemptive vs nonpreemptive; menyebutkan 5 kriteria penjadwalan dan arti tiap kriteria	PG / Essay
21	Algoritma FCFS, SJF, RR, Priority	Menggambar Gantt chart dan **menghitung average waiting time** untuk tiap algoritma; menjelaskan starvation dan aging	**Hitungan** + essay

> **Prioritas belajar:** Topik nomor 16, 20, dan 21 berisi **perhitungan**, jadi latihannya perlu dikerjakan sendiri, bukan hanya dibaca. Topik lain cukup dipahami dengan membuat ringkasan singkat dengan kata-katamu sendiri.

B. Penjelasan Materi

B.1 Pengantar Sistem Operasi (Bab 1)

Apa itu Sistem Operasi?

Sistem operasi (OS) adalah **program perantara** antara pengguna (user) dan perangkat keras (hardware) komputer. Tiga tujuannya:

1. Menjalankan program user dan memudahkan user menyelesaikan masalahnya.
2. Membuat komputer nyaman dipakai.
3. Memakai hardware secara efisien.

> **Analogi:** OS seperti *manajer gedung*. Penyewa (program) boleh memakai ruangan (CPU, memori, printer), tapi manajer yang mengatur siapa dapat giliran dan mencegah kekacauan.

Empat komponen sistem komputer

Komponen	Penjelasan	Contoh
Hardware	Menyediakan sumber daya komputasi dasar: CPU, memori, perangkat I/O	CPU, RAM, disk, keyboard
Sistem operasi	Mengontrol dan mengoordinasi penggunaan hardware oleh berbagai program	Windows, Linux, macOS
Program aplikasi	Menentukan cara sumber daya dipakai untuk menyelesaikan masalah user	Word processor, browser, compiler
User	Orang, mesin, atau komputer lain yang memakai sistem	Kamu, server lain

Kernel, system program, dan application program

- **Kernel**: inti OS yang **selalu berjalan** dari booting sampai shutdown.
- **System program**: ikut OS, tapi bukan kernel. Contoh: `ls`, `top`, `dir`.
- **Application program**: semua program di luar OS. Contoh: browser, game.

Interrupt, trap, dan exception

- **Interrupt**: sinyal dari *hardware* (misalnya device selesai bekerja) yang membuat CPU menghentikan sementara pekerjaannya dan langsung menuju **ISR (Interrupt Service Routine)**.
- **Interrupt vector**: tabel berisi alamat semua ISR.
- **Trap / exception**: interrupt dari *software*, akibat error (misalnya pembagian dengan nol) atau permintaan user (system call).
- Sebelum menangani interrupt, OS **menyimpan state CPU** (register dan program counter) agar bisa dilanjutkan nanti.

> **Ingat:** OS bersifat *interrupt driven*, artinya OS bekerja berdasarkan sinyal interrupt dari hardware maupun software.

Dual mode: user mode dan kernel mode

Hardware menyediakan **mode bit** untuk melindungi OS. Dalam materi ini: **1 = user mode**, **0 = kernel mode**.

- Program user berjalan di **user mode**. Instruksi *privileged* (berbahaya) tidak boleh dijalankan di sini.
- Kode OS berjalan di **kernel mode** dan boleh menjalankan instruksi privileged.
- Saat **system call** dipanggil, mode berpindah ke kernel. Saat system call selesai, mode kembali ke user.

Timer

Timer mencegah program user terjebak *infinite loop* atau menguasai CPU terlalu lama. OS mengatur counter timer (instruksi privileged); counter ini berkurang setiap detak clock, dan saat mencapai nol timer memicu interrupt sehingga OS mendapatkan kembali kendali CPU.

Hierarki penyimpanan & caching

Penyimpanan diurutkan dari **cepat, mahal, dan kecil** ke **lambat, murah, dan besar**:

```
Register → Cache → Main memory (RAM, volatile) → Secondary storage (SSD/HDD, non-volatile)
```

Caching berarti menyalin data yang sering dipakai ke penyimpanan yang lebih cepat. Cache selalu lebih kecil dari yang di-cache-nya, sehingga pengelolaan *ukuran* dan *kebijakan penggantian* (replacement policy) menjadi masalah desain utama.

Satuan: 1 byte = 8 bit. 1 KB = 1.024 byte (bukan 1.000).

Multiprogramming vs multitasking (time-sharing)

	Multiprogramming (batch)	Multitasking (time-sharing)
Tujuan	Agar CPU selalu punya pekerjaan	Agar user bisa berinteraksi dengan program yang sedang berjalan
Cara	Ketika satu job menunggu I/O, CPU pindah ke job lain	CPU berpindah job sangat sering sehingga terasa paralel
Target	Utilisasi CPU tinggi	Response time < 1 detik

Pengelolaan I/O & DMA

- OS menyembunyikan kerumitan hardware lewat **device driver** dan **I/O subsystem** (buffering, caching, spooling).
- **Spooling**: menumpuk antrean pekerjaan output, misalnya antrean printer.
- **DMA (Direct Memory Access)**: controller memindahkan **blok data** langsung ke memori tanpa CPU, sehingga hanya **satu interrupt per blok** (bukan per byte).

Booting (startup)

Urutan booting: **BIOS/UEFI** (firmware di ROM/EPROM, menjalankan POST) → **bootloader** (misal GRUB) memuat kernel ke RAM → **kernel** menginisialisasi hardware dan driver → **system init** (systemd/init, PID 1) menjalankan daemon.

Sistem terdistribusi, multiprosesor, dan cluster

- **Distributed system**: kumpulan komputer terhubung jaringan (LAN, MAN, WAN, PAN) yang tampak seperti satu sistem.
- **Multiprosesor (SMP)** mengikuti dua model. *Asymmetric*: tiap CPU punya tugas khusus. *Symmetric*: tiap CPU mengerjakan semua jenis tugas. Keuntungan: throughput lebih tinggi, hemat biaya, lebih andal (*graceful degradation*).
- **Clustered system**: beberapa komputer bekerja bersama, biasanya memakai storage bersama (SAN), untuk ketersediaan tinggi (*high availability*).

Virtualisasi & cloud

- **Virtualisasi**: menjalankan OS lain (guest) di atas OS utama (host) lewat **VMM (Virtual Machine Manager)**.
- **Cloud**: layanan komputasi lewat internet. Jenisnya: **SaaS** (aplikasi siap pakai), **PaaS** (platform untuk developer), **IaaS** (server/storage). Modelnya: public, private, hybrid.
- **Real-time OS**: punya batas waktu tetap. Hasil dianggap benar hanya jika batas waktu terpenuhi.

Open source vs free software

Keduanya berbeda. **Free software** dipelopori FSF dengan lisensi *copyleft* GPL. **Open source** menekankan akses kode sumber. Contoh: GNU/Linux, BSD UNIX.

B.2 Layanan, System Call, dan Struktur OS (Bab 2)

Layanan OS

Untuk user	Untuk efisiensi sistem
• Eksekusi program (memuat, menjalankan, mengakhiri) • Operasi I/O • Manajemen file • Komunikasi antar-proses (shared memory / message passing) • Deteksi error	• **Resource allocation**: membagi CPU, memori, storage antar user/job • **Accounting**: mencatat pemakaian resource • **Protection**: mengontrol akses ke resource • **Security**: autentikasi, enkripsi, integritas data

Antarmuka user: **CLI** (perintah teks, bisa di-scripting) dan **GUI** (window, icon, menu, pointer, touchscreen).

System call

System call adalah **cara program user meminta layanan kernel**. Biasanya dipanggil lewat **API** (Win32 API, POSIX API, Java API) agar kode lebih portabel.

```
User mode (1) → system call → Kernel mode (0): jalankan layanan → kembali ke User mode (1)
```

Kategori	Fungsi	Contoh (POSIX / Windows)
Process control	Membuat, menghentikan, mengatur proses	`fork()`, `exit()` / `CreateProcess()`

File management	Membuka, membaca, menulis, menutup file	`open()`, `read()` / `CreateFile()`
Device management	Meminta, membaca, menulis ke perangkat	`ioctl()`, `read()` / `SetConsoleMode()`
Information	Mengambil/mengatur waktu, tanggal, data sistem	`getpid()`, `time()` / `GetSystemTime()`
Communications	Membuat koneksi dan mengirim pesan	`pipe()`, `socket()` / `CreatePipe()`

Struktur kernel (perbandingan)

Arsitektur	Keunggulan	Kelemahan	Contoh
Monolitik	Sangat cepat (tanpa overhead pergantian mode antar-layanan)	Satu bug (misal di driver) bisa membuat seluruh sistem crash	MS-DOS, UNIX awal
Layered	Modul terisolasi, mudah di-debug	Penentuan urutan lapisan sulit; banyak overhead antar-lapisan	THE OS, OS/2 awal
Microkernel	Kernel minimal (memori dasar, proses minimal, IPC) → aman dan mudah diperluas	Overhead komunikasi IPC tinggi	Mach, QNX, L4
Modular (LKM)	Kernel inti + modul yang dimuat saat dibutuhkan; fleksibel	Pengelolaan dependensi antar-modul rumit	Linux, Solaris
Hybrid	Menggabungkan performa dan fleksibilitas	Struktur internal kompleks	macOS, iOS, Windows 11

> **Cara mengingat:** *Monolitik = semua dalam satu wadah. Layered = bertingkat. Microkernel = kernel dikecilkan, sisanya di user space. LKM = inti kecil + modul yang bisa dimasukkan saat jalan. Hybrid = campuran.*

B.3 Konsep Proses

Program vs proses

Program (entitas pasif)	Proses (entitas aktif)
Kumpulan instruksi yang tersimpan di disk (file executable)	Program yang **sedang dieksekusi**
Belum memakai CPU atau RAM	Memiliki memori aktif, register CPU, dan status eksekusi

> **Analogi:** Program = *resep masakan* yang tertulis di buku. Proses = *kegiatan memasak* dengan resep itu, lengkap dengan kompor, panci, dan bahan yang sedang dipakai.

Struktur memori proses

Bagian	Isi
Text	Kode instruksi program (umumnya read-only)
Data	Variabel global dan statis
Heap	Memori yang dialokasikan saat run-time (`malloc()`, `new`)
Stack	Parameter fungsi, return address, variabel lokal

Status proses (5 status)

Status	Arti
New	Proses sedang dibuat oleh OS
Ready	Siap dan menunggu CPU
Running	Instruksinya sedang dieksekusi CPU
Waiting	Menunggu suatu kejadian, misalnya selesainya I/O
Terminated	Eksekusi selesai

Alur perpindahan status:

- New → Ready: proses diterima (*admitted*).
- Ready → Running: dipilih penjadwal (*dispatch*).
- Running → Ready: waktu habis (interrupt timer) atau di-*preempt*.
- Running → Waiting: meminta I/O atau menunggu event.
- Waiting → Ready: I/O sudah selesai.
- Running → Terminated: proses selesai (`exit()`).

Process Control Block (PCB)

PCB adalah "kartu identitas" setiap proses yang disimpan OS. Isinya:

- **Identitas & eksekusi**: process state, PID, program counter, CPU registers
- **Penjadwalan**: prioritas dan pointer ke antrean
- **Memori**: base/limit register atau page table
- **Accounting**: waktu CPU yang dipakai, batas waktu
- **I/O**: daftar perangkat I/O dan file yang terbuka

Antrean & penjadwal (scheduler)

Antrean	Isi
Job queue	Semua proses baru yang ada di storage sekunder
Ready queue	Proses di memori utama yang siap dan menunggu CPU
Device queue	Proses yang menunggu perangkat I/O tertentu

Scheduler	Frekuensi	Fungsi
Short-term (CPU scheduler)	Milidetik	Memilih proses dari ready queue untuk diberi CPU. Harus sangat cepat.

Long-term (job scheduler)	Detik/menit	Memilih job dari disk ke memori. Mengatur **derajat multiprogramming**.
Medium-term	Kondisional	Swapping: memindahkan proses dari RAM ke disk sementara untuk menghemat memori.

Context switch

Saat CPU berpindah dari proses A ke proses B, OS **menyimpan state A ke PCB-nya**, lalu **memuat state B dari PCB-nya**. Waktu ini adalah **overhead murni**, karena CPU tidak mengerjakan hal yang berguna.

Pembuatan & penghentian proses (Unix/Linux)

- Proses membentuk pohon **parent-child**. Setiap proses punya **PID** unik.
- `fork()`: menduplikasi proses induk menjadi proses anak.
- `exec()`: mengganti isi memori proses dengan program baru (biasanya dipanggil setelah `fork()`).
- `exit()`: menghentikan proses dan mengembalikan status ke parent lewat `wait()`.
- `abort()`: parent menghentikan child, misalnya karena tugasnya tidak lagi diperlukan atau melewati kuota.
- **Zombie process**: child sudah selesai, tapi parent belum memanggil `wait()`.
- **Orphan process**: parent sudah mati lebih dulu, sehingga child kehilangan induknya.

B.4 Komunikasi Antar-Proses (IPC) & Client-Server

Mengapa perlu IPC?

Proses **independent** tidak memengaruhi dan tidak dipengaruhi proses lain. Proses **cooperating** bekerja sama, dengan alasan: **berbagi informasi**, **mempercepat komputasi**, **modularitas**, dan **kenyamanan**.

Shared memory vs message passing

Aspek	Shared memory	Message passing
Mekanisme	Area memori bersama diakses banyak proses	Pesan dikirim lewat kernel OS
Kecepatan	Sangat cepat (tanpa kernel)	Lebih lambat (butuh system call)
Sinkronisasi	Diatur programmer (rawan konflik)	Diatur otomatis oleh OS
Cocok untuk	Komputasi lokal, data besar	Sistem terdistribusi, data kecil

Producer-consumer

Producer membuat data, consumer memakainya. Lewat **bounded buffer** (buffer berukuran terbatas), producer menunggu jika buffer penuh dan consumer menunggu jika buffer kosong. Lewat **unbounded buffer**, ukurannya tak terbatas sehingga producer tidak pernah menunggu. Masalah utamanya: dua proses tidak boleh menulis ke lokasi yang sama secara bersamaan.

Message passing: detail

- **Direct**: pengirim dan penerima menyebut nama langsung, misalnya `send(P, message)` dan `receive(Q, message)`.
- **Indirect**: pesan lewat **mailbox/port**, misalnya `send(A, message)`.
- **Blocking**: pengirim/penerima menunggu sampai pesan selesai. **Non-blocking**: proses langsung lanjut.

- **Kapasitas buffer**: *zero capacity* (pengirim menunggu penerima siap, disebut *rendezvous*), *bounded* (pengirim menunggu jika antrean penuh), *unbounded* (pengirim tidak pernah diblokir).

Komunikasi client-server

- **Socket**: endpoint jaringan, gabungan **IP address + port**. Port di bawah 1024 untuk layanan standar (80 = HTTP, 22 = SSH).
  - **TCP**: connection-oriented, andal.
  - **UDP**: connectionless, cepat, cocok untuk data real-time.
- **RPC (Remote Procedure Call)**: memanggil fungsi di komputer lain seolah-olah fungsi lokal. Parameter dibungkus (*marshalling*) oleh **stub** sebelum dikirim.
- **Pipe**: saluran yang menghubungkan dua proses.
  - **Ordinary pipe**: satu arah, harus ada hubungan parent-child.
  - **Named pipe (FIFO)**: dua arah, tanpa hubungan parent-child.

B.5 Thread & Multithreading (Bab 4)

Apa itu thread?

Thread adalah **unit dasar pemakaian CPU**. Setiap thread punya **Thread ID, program counter, register set, dan stack** sendiri. Thread dalam satu proses **berbagi code, data, dan resource** (misalnya file yang terbuka).

> **Analogi:** Satu proses = satu *dapur*. Thread = *koki* di dapur itu. Tiap koki punya pisau dan catatan resepnya sendiri (stack dan PC), tapi mereka memakai kulkas dan bumbu yang sama (code, data, file).

Single-threaded vs multithreaded

Single-threaded	Multithreaded
Hanya punya satu alur eksekusi. Jika ada blocking system call, seluruh proses ikut berhenti.	Punya beberapa alur eksekusi dalam satu address space. Bisa mengerjakan beberapa tugas sekaligus (misalnya UI update sambil mengunduh data).
Satu PC, satu set register, satu stack	Tiap thread punya PC dan stack sendiri, tapi berbagi code, data, dan file

Empat keuntungan multithreading

1. **Responsiveness**: aplikasi tetap interaktif walau sebagian thread sedang menunggu.
2. **Resource sharing**: thread berbagi memori tanpa butuh IPC.
3. **Economy**: membuat thread jauh lebih murah dan cepat daripada membuat proses.
4. **Scalability**: thread bisa berjalan paralel di banyak core.

Concurrency vs parallelism

Concurrency	Parallelism
Beberapa tugas **sama-sama maju** dengan cara bergantian (interleaving). Bisa terjadi di satu core.	Beberapa tugas **benar-benar berjalan bersamaan** di core berbeda. Butuh hardware multicore.

> **Ingat:** Setiap parallelism adalah concurrency, tetapi tidak sebaliknya. Concurrency adalah soal *struktur* (banyak tugas yang maju), parallelism adalah soal *eksekusi simultan*.

Data parallelism vs task parallelism

- **Data parallelism**: data dibagi, lalu tiap core menjalankan **operasi yang sama** pada bagian data yang berbeda. Contoh: mengalikan elemen matriks besar secara simultan.
- **Task parallelism**: tugas dibagi, lalu tiap thread menjalankan **operasi yang berbeda**. Contoh: thread 1 mengedit UI, thread 2 mengunduh file, thread 3 memeriksa ejaan.

Hukum Amdahl

Hukum ini menghitung speedup yang bisa didapat dengan menambah core, jika program punya bagian serial dan bagian paralel.

```
Speedup = 1 / ( S + (1 − S) / N )
S = porsi serial, N = jumlah core. Jika N → ∞, speedup → 1 / S
```

Contoh: program 75% paralel (S = 0,25). Dari 1 ke 2 core: 1 / (0,25 + 0,75/2) = **1,6 kali**. Walaupun core ditambah terus, speedup maksimum hanya 1/0,25 = **4 kali**. Jadi bagian serial sangat memengaruhi hasil.

User thread vs kernel thread

User thread	Kernel thread
Dikelola thread library di user level, tanpa dukungan kernel. Pembuatan dan pengelolaannya sangat cepat.	Dikelola dan didukung langsung oleh OS. Jika satu thread terblokir, kernel bisa menjadwalkan thread lain dari proses yang sama.
Jika satu user thread terblokir, **seluruh proses ikut terblokir**	Lebih lambat dibuat dan dikelola (butuh system call)

Model pemetaan multithreading

Model	Penjelasan	Kelebihan / kekurangan
Many-to-one	Banyak user thread → 1 kernel thread	Efisien, tapi satu thread terblokir membuat semua ikut terblokir; tidak bisa memakai multicore
One-to-one	1 user thread → 1 kernel thread	Lebih konkuren dan mendukung multicore; dipakai Windows dan Linux
Many-to-many	M user thread → N kernel thread (M ≥ N)	Paling fleksibel, tetapi implementasinya paling rumit

Thread library

- **POSIX Pthreads**: standar IEEE (POSIX.1c). `pthread_create()` membuat thread, `pthread_join()` menunggu thread anak selesai. Data yang dipakai bersama biasanya berupa variabel global.
- **Windows threads**: library kernel-level pada Windows, memakai Win32/Win64 API (`CreateThread()`).
- **Java threads**: dibuat dengan *extend* kelas `Thread` atau *implement* interface `Runnable`. Kode tugas ada di `run()`, eksekusi dimulai dengan `start()`, dan koordinasi lewat `join()`. Versi modern memakai `ExecutorService`.

Implicit threading

Pembuatan dan pengelolaan thread diserahkan ke compiler atau run-time library.

- **Thread pool**: sejumlah thread dibuat di awal dan menunggu tugas. Tugas dimasukkan ke antrean, lalu thread yang bebas mengambilnya dan kembali ke pool setelah selesai. *Manfaat*: lebih cepat (thread dipakai ulang), jumlah thread terbatas, dan pembuatan tugas terpisah dari eksekusinya.
- **OpenMP**: kumpulan compiler directive dan API untuk C/C++ yang otomatis memparalelkan blok kode.
- **Grand Central Dispatch (GCD)**: teknologi Apple (macOS/iOS) yang mengelola *dispatch queue* tugas di level sistem.

Isu-isu multithreading

- **fork() dan exec()**: `fork()` punya dua versi, yaitu menduplikasi *semua* thread atau hanya *thread pemanggil*. Jika `exec()` langsung dipanggil setelah `fork()`, cukup duplikasi thread pemanggil saja, karena `exec()` akan mengganti seluruh proses.
- **Thread cancellation**:
  - *Asynchronous*: thread target langsung dihentikan. Risiko: data bersama bisa rusak jika thread sedang menulis atau memegang lock.
  - *Deferred*: thread target memeriksa sendiri di *cancellation point* apakah harus berhenti, sehingga bisa membersihkan resource dengan aman.
- **Signal handling**: sinyal UNIX memberi tahu proses bahwa suatu peristiwa terjadi (bisa *synchronous* atau *asynchronous*). Sinyal multithread dapat dikirim ke: thread asal sinyal, semua thread, thread tertentu, atau thread khusus penangan sinyal.
- **Thread-local storage (TLS)**: setiap thread punya salinan data pribadinya. Bedanya dengan variabel lokal: variabel lokal hanya hidup selama fungsi dipanggil, TLS hidup selama thread berjalan.

Implementasi di OS nyata

Fitur	Windows	Linux (NPTL)
Model	One-to-one	One-to-one
Struktur data	ETHREAD (executive) & KTHREAD (kernel)	`struct task_struct`
Pembuatan	`CreateThread()`	System call `clone()`
Istilah	Process dan thread dibedakan jelas	Thread disebut *task* atau *light weight process*

B.6 CPU Scheduling (Bab 5)

Kapan penjadwal memilih proses?

1. Running → Waiting (**tidak ada pilihan**, harus pilih proses baru)
2. Running → Ready (**ada pilihan**)
3. Waiting → Ready (**ada pilihan**)
4. Terminate (**tidak ada pilihan**)

- **Nonpreemptive**: penjadwalan hanya terjadi di situasi 1 dan 4. Proses menguasai CPU sampai ia melepasnya sendiri.
- **Preemptive**: penjadwalan juga terjadi di situasi 2 dan 3. CPU bisa diambil dari proses yang sedang berjalan. Hampir semua OS modern (Windows, macOS, Linux, UNIX) memakai preemptive.

Lima kriteria penjadwalan

Kriteria	Tujuan	Arti
CPU utilization	Maksimum	Seberapa sibuk CPU
Throughput	Maksimum	Jumlah proses yang selesai per satuan waktu
Turnaround time	Minimum	Waktu dari proses masuk sampai selesai (Finish − Arrival)
Waiting time	Minimum	Total waktu proses menunggu di ready queue (Turnaround − Burst)
Response time	Minimum	Waktu dari request dikirim sampai respons **pertama** muncul

Istilah wajib: arrival time, burst time

Burst time adalah lama proses memakai CPU. **Arrival time** adalah saat proses masuk ready queue. Jika soal tidak menyebut arrival time, anggap semua proses datang pada **t = 0**.

Algoritma penjadwalan

Algoritma	Cara kerja	Kelebihan / kekurangan
FCFS First-Come, First-Served	Proses yang datang duluan dilayani duluan. Nonpreemptive.	Sederhana. Kekurangan: *convoy effect*, yaitu proses pendek ikut menunggu di belakang proses panjang.
SJF Shortest Job First	Dipilih proses dengan *burst berikutnya* terpendek.	**Optimal**: menghasilkan rata-rata waiting time terkecil. Kekurangan: panjang burst berikutnya harus *diperkirakan*. Versi preemptive disebut **SRTF**.
Round Robin (RR)	Tiap proses dapat jatah waktu kecil (**time quantum q**, biasanya 10–100 ms). Jika belum selesai, proses dipindah ke akhir antrean. Preemptive.	Respons lebih baik. Jika q besar → mirip FCFS. Jika q kecil → banyak context switch. Maka q harus besar dibanding waktu context switch. Setiap proses tidak menunggu lebih dari (n−1)q.
Priority	Proses dengan **angka prioritas terkecil** dipilih lebih dulu. Bisa preemptive atau nonpreemptive.	Kekurangan: *starvation* (proses prioritas rendah tidak pernah dijalankan). Solusi: **aging**, yaitu prioritas naik seiring waktu tunggu.

> **Catatan penting:** SJF adalah kasus khusus priority scheduling, di mana prioritas = kebalikan dari burst berikutnya. Priority + RR: proses prioritas tertinggi jalan dulu; jika ada beberapa proses dengan prioritas sama, mereka bergantian dengan RR.

C. Latihan Soal & Pembahasan

Bagian ini berisi contoh soal dengan pembahasan langkah demi langkah. Kerjakan dulu sendiri, baru cocokkan dengan pembahasan.

C.1 Soal Konseptual

1. Sebutkan 3 tujuan sistem operasi dan jelaskan fungsi kernel.

Jawab: Tiga tujuan: (1) menjalankan program user dan memudahkan user menyelesaikan masalah, (2) membuat komputer nyaman dipakai, (3) memakai hardware secara efisien. Kernel adalah bagian inti OS yang selalu berjalan sejak booting sampai shutdown.

2. Apa perbedaan interrupt dan trap? Berikan contoh masing-masing.

Jawab: Interrupt berasal dari *hardware* (contoh: disk selesai membaca data). Trap berasal dari *software*, karena error (contoh: pembagian dengan nol) atau permintaan user (system call).

3. Jelaskan apa yang terjadi saat program user memanggil system call. Sebutkan mode bit sebelum dan sesudahnya.

Jawab: (1) Program berada di user mode (mode bit = 1). (2) Pemanggilan system call memindahkan eksekusi ke kernel mode (mode bit = 0). (3) Kernel menjalankan layanan yang diminta. (4) Setelah selesai, mode bit kembali ke 1 (user). Dengan cara ini, program user tidak bisa langsung menjalankan instruksi privileged.

4. Mengapa sistem memakai caching? Apa hubungannya dengan hierarki penyimpanan?

Jawab: Penyimpanan cepat itu kecil dan mahal, sedangkan penyimpanan besar itu lambat. Caching menyalin data yang sering dipakai ke penyimpanan yang lebih cepat, sehingga akses berikutnya lebih cepat. Cache adalah bagian dari hierarki: register → cache → RAM → disk.

5. Sebutkan 5 status proses dan gambarkan alur perpindahannya.

Jawab: New, Ready, Running, Waiting, Terminated. Alur umum: New → Ready → Running → (Waiting → Ready → Running)* → Terminated. Running kembali ke Ready jika waktu habis atau di-preempt.

6. Apa perbedaan zombie process dan orphan process?

Jawab: Zombie: child sudah selesai, tetapi parent belum memanggil `wait()` untuk mengambil status exit-nya. **Orphan**: parent mati lebih dulu, sehingga child kehilangan induknya.

7. Bandingkan shared memory dan message passing dalam 3 aspek.

Jawab: (1) *Kecepatan*: shared memory lebih cepat karena tanpa kernel, message passing lebih lambat karena lewat system call. (2) *Sinkronisasi*: shared memory diatur programmer, message passing diatur OS. (3) *Kecocokan*: shared memory cocok untuk komputasi lokal, message passing cocok untuk sistem terdistribusi.

8. Sebuah program 80% paralel dijalankan di 4 core. Hitung speedup-nya dan jelaskan batas maksimumnya.

Jawab: S = 0,2 dan N = 4.
Speedup = 1 / (0,2 + 0,8/4) = 1 / (0,2 + 0,2) = 1 / 0,4 = **2,5 kali**.
Batas maksimum (N → ∞) = 1 / S = 1 / 0,2 = **5 kali**, walaupun core ditambah tak terbatas.

9. Mengapa many-to-one model tidak bisa memanfaatkan multicore? Dan apa kelemahan utamanya?

Jawab: Pada many-to-one, banyak user thread dipetakan ke **satu** kernel thread. Hanya satu thread yang bisa berjalan di kernel pada satu waktu, sehingga tidak ada paralelisme di multicore. Kelemahan utamanya: jika satu thread terblokir (misalnya menunggu I/O), seluruh proses ikut terblokir.

10. Mengapa asynchronous cancellation berisiko, dan bagaimana deferred cancellation mengatasinya?

Jawab: Asynchronous cancellation menghentikan thread seketika, sehingga bisa terjadi saat thread sedang mengubah data bersama atau memegang lock, yang membuat data tidak konsisten. Deferred cancellation membiarkan thread memeriksa sendiri di cancellation point, sehingga ia bisa membersihkan resource dan menyimpan data dengan aman sebelum berhenti.

C.2 Soal Hitungan CPU Scheduling (Contoh dari Materi)

Asumsi: semua proses tiba pada t = 0 dan tidak ada I/O. Rumus yang dipakai:
Turnaround = Finish − Arrival | **Waiting = Turnaround − Burst** | **Average = jumlah waiting ÷ jumlah proses**

Contoh 1 – FCFS

Proses: P1 (burst 24), P2 (burst 3), P3 (burst 3). Urutan kedatangan: P1, P2, P3.

P1 (0–24)	P2 (24–27)	P3 (27–30)

Waiting: P1 = 0, P2 = 24, P3 = 27. Rata-rata = (0 + 24 + 27) / 3 = **17**.

Jika urutannya diganti menjadi P2, P3, P1, Gantt-nya: P2 (0–3), P3 (3–6), P1 (6–30). Waiting: P2 = 0, P3 = 3, P1 = 6. Rata-rata = **3**. Perbedaan besar ini adalah *convoy effect*: proses pendek lebih baik didahulukan.

Contoh 2 – SJF (nonpreemptive)

Proses: P1 (6), P2 (8), P3 (7), P4 (3). Urutkan berdasarkan burst terkecil: P4 (3), P1 (6), P3 (7), P2 (8).

P4 (0–3)	P1 (3–9)	P3 (9–16)	P2 (16–24)

Waiting: P4 = 0, P1 = 3, P3 = 9, P2 = 16. Total = 28. Rata-rata = 28 / 4 = **7**.

Contoh 3 – Round Robin (q = 4)

Proses: P1 (24), P2 (3), P3 (3). Urutan antrean: P1, P2, P3.

P1 0–4	P2 4–7	P3 7–10	P1 10–14	P1 14–18	P1 18–22	P1 22–26	P1 26–30

Finish: P2 = 7, P3 = 10, P1 = 30. Waiting = Finish − Burst: P1 = 30 − 24 = 6, P2 = 7 − 3 = 4, P3 = 10 − 3 = 7. Rata-rata waiting = (6 + 4 + 7) / 3 = **5,67**.

> **Perbandingan:** Dengan urutan yang sama, FCFS menghasilkan rata-rata 17, sedangkan RR menghasilkan 5,67. Selain itu, response time P2 dengan RR hanya 4, dibanding 24 dengan FCFS. Itulah keunggulan RR: respons lebih cepat.

Contoh 4 – Priority (nonpreemptive; angka kecil = prioritas tinggi)

Proses	Burst	Prioritas
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

Urutan prioritas: P2 (1), P5 (2), P1 (3), P3 (4), P4 (5).

P2 (0–1)	P5 (1–6)	P1 (6–16)	P3 (16–18)	P4 (18–19)

Waiting: P2 = 0, P5 = 1, P1 = 6, P3 = 16, P4 = 18. Total = 41. Rata-rata = 41 / 5 = **8,2**.

C.3 Soal Latihan (kerjakan sendiri, lalu cek jawaban)

Soal A. Proses P1 (burst 8), P2 (burst 4), P3 (burst 9), P4 (burst 5). Semua tiba pada t = 0 dengan urutan P1, P2, P3, P4. Hitung rata-rata waiting time untuk FCFS, SJF, dan RR (q = 3).

FCFS: P1 (0–8), P2 (8–12), P3 (12–21), P4 (21–26). Waiting: 0, 8, 12, 21. Total = 41. Rata-rata = **10,25**.

SJF: urutan P2 (4), P4 (5), P1 (8), P3 (9). Gantt: P2 (0–4), P4 (4–9), P1 (9–17), P3 (17–26). Waiting: 0, 4, 9, 17. Total = 30. Rata-rata = **7,5**.

RR (q = 3): urutan kejadian: P1 (0–3), P2 (3–6), P3 (6–9), P4 (9–12), P1 (12–15), P2 (15–16, selesai), P3 (16–19), P4 (19–21, selesai), P1 (21–23, selesai), P3 (23–26, selesai).
Finish: P1 = 23, P2 = 16, P3 = 26, P4 = 21. Waiting: P1 = 15, P2 = 12, P3 = 17, P4 = 16. Total = 60. Rata-rata = **15**.

Catatan: SJF memberi waiting time terkecil (terbukti optimal). RR menghasilkan waiting time lebih besar, tetapi response time pertamanya lebih cepat.

Soal B. Sebuah aplikasi 90% paralel dijalankan pada 8 core. Hitung speedup. Berapa batas maksimum speedup-nya?

S = 0,1 dan N = 8. Speedup = 1 / (0,1 + 0,9/8) = 1 / (0,1 + 0,1125) = 1 / 0,2125 ≈ **4,7 kali**. Batas maksimum = 1 / 0,1 = **10 kali**.

Soal C. Jelaskan perbedaan user thread dan kernel thread, lalu sebutkan model pemetaan yang paling tepat untuk sistem multicore dan alasannya.

User thread dikelola library di user level (cepat, tapi satu thread terblokir memblokir seluruh proses). Kernel thread dikelola OS (lebih lambat, tapi thread lain tetap bisa jalan). Untuk multicore, model **one-to-one** (dipakai Windows dan Linux) atau **many-to-many** paling tepat, karena keduanya bisa memanfaatkan banyak core. Many-to-one tidak bisa.

D. Glosarium Istilah Penting

Istilah	Bahasa Indonesia	Penjelasan singkat
Operating system (OS)	Sistem operasi	Program perantara antara user dan hardware
Kernel	Inti OS	Bagian OS yang selalu berjalan
Interrupt	Interupsi	Sinyal dari hardware yang menghentikan sementara CPU
Trap / exception	Jebakan / pengecualian	Interrupt dari software (error atau system call)
Mode bit	Bit mode	Penanda user mode (1) atau kernel mode (0)
System call	Panggilan sistem	Permintaan program user ke layanan kernel
Caching	Caching	Menyalin data ke penyimpanan yang lebih cepat

DMA	Akses memori langsung	Device memindahkan blok data ke RAM tanpa CPU
Spooling	Spooling	Menumpuk antrean pekerjaan output (misal printer)
Process (proses)	Proses	Program yang sedang dieksekusi (entitas aktif)
PCB	Blok kontrol proses	Struktur data OS yang menyimpan informasi tiap proses
Context switch	Pergantian konteks	Menyimpan state proses lama dan memuat state proses baru
Zombie / orphan	Zombie / yatim	Child selesai tanpa diambil parent / child yang kehilangan parent
IPC	Komunikasi antar-proses	Mekanisme pertukaran data antar proses
Socket	Soket	Endpoint komunikasi jaringan (IP + port)
RPC	Pemanggilan prosedur jarak jauh	Memanggil fungsi di komputer lain seolah lokal
Thread	Utas	Unit dasar eksekusi CPU di dalam proses
Concurrency	Konkurensi	Banyak tugas maju secara bergantian
Parallelism	Paralelisme	Banyak tugas berjalan bersamaan di banyak core
Amdahl's law	Hukum Amdahl	Rumus speedup berdasarkan porsi serial dan paralel
Thread pool	Kumpulan thread	Thread yang dibuat di awal dan dipakai ulang
TLS	Penyimpanan lokal thread	Data pribadi tiap thread yang hidup selama thread berjalan
Scheduler	Penjadwal	Komponen OS yang memilih proses untuk CPU
Preemptive	Dapat dipotong	CPU bisa diambil dari proses yang sedang berjalan
Burst time	Waktu burst	Lama proses memakai CPU
Arrival time	Waktu kedatangan	Saat proses masuk ready queue
Turnaround time	Waktu penyelesaian	Finish − Arrival
Waiting time	Waktu tunggu	Waktu proses menunggu di ready queue
Response time	Waktu respons	Waktu dari request sampai respons pertama
Time quantum	Jatah waktu	Lama maksimum satu proses memakai CPU pada RR
Starvation	Kelaparan	Proses tidak pernah mendapat CPU karena prioritasnya terus dikalahkan
Aging	Penuaan prioritas	Prioritas proses naik seiring waktu tunggu, untuk mencegah starvation

Tips terakhir: Hafalkan rumus Amdahl dan rumus waiting time, lalu latihan Gantt chart FCFS, SJF, RR, dan Priority sampai bisa mengerjakannya tanpa melihat contoh. Untuk soal essay, gunakan pola *apa → bagaimana cara kerjanya → kelebihan dan kekurangan → contoh*.